

第17課

WiFiモード

導入

このセクションでは、ESP32シリーズチップ（ESP32-C6などのモデルを含む）のWi-Fiコア機能について体系的に調査します。まず、ステーション（STA）、ソフトAP（AP）、STA+AP共存という3つのコア動作モードの技術的口ジックとアプリケーションシナリオを含め、Wi-Fi通信の基本的な原理を深く分析します。

ESP32 Wi-Fi機能の概要

高度に統合された無線通信機能を備えたESP32シリーズのチップは、モノのインターネット（IoT）、スマートホーム、産業オートメーションなどの分野において、中核となる有力なソリューションとなっています。

このシリーズのほとんどのモデルは高性能な無線モジュールを搭載しており、中でも2.4GHz Wi-Fiは基本規格として採用されているため、ほとんどの低消費電力・短距離データ伝送シナリオのニーズを満たすことができます。

ESP32-C6に代表される新モデルは、ワイヤレス機能の限界をさらに拡大し、5GHzデュアルバンドWi-Fiをサポートするだけでなく、新しいWi-Fi 6（802.11ax）プロトコルにも対応し、さらに802.11b/g/nなどの従来の規格にも対応しています。

これらは、より高い伝送速度、より低いネットワーク遅延、そして複数のデバイス間でのより安定した同時接続体験を提供できます。

注目すべきは、ESP32ファミリーの一部の特殊モデル（PシリーズやHシリーズなど）では、より豊富なコンピューティングリソースとより安定した産業レベルのパフォーマンスを実現するために、Wi-Fi機能モジュールを簡素化し、高性能コンピューティングや産業レベルの制御といった特定のシナリオに特化している点である。

各モデルがサポートする無線機能の詳細は、Espressif社が公開している公式文書「ESP32製品概要」に基づいて正確に選択できます。

ESP32シリーズチップのWi-Fi機能は、幅広い動作モードを提供し、さまざまな開発要件に柔軟に対応できます。

- **ステーション (STA)モード** :チップはWi-Fiクライアントとして動作し、既存のネットワークにアクティブに接続します。
ルーターやホットスポットなどの無線ネットワーク、動的IPアドレスの取得、ローカルネットワークへの参加、外部ネットワーク内またはローカルネットワーク内のデバイスとのデータ通信の実現。
- **SoftAP (ソフトAP)モード** :チップが独立したWi-Fiホットスポットを独自に作成し、携帯電話、コンピュータ、その他のシングルチップマイクロコントローラなどのデバイスが接続できるようにします。これにより、外部ルーターを必要とせずにポイントツーポイントまたは小規模なローカルエリアネットワークが構築され、ローカルデバイスのデバッグ、オフラインでのデータやり取りなどに適しています。
- **STA + AP共存モード** :このチップは両方のモードを同時に実行します。外部ネットワークに接続してデータアップロードやリモート制御を実現できるだけでなく、デバッグ機器を接続するためのローカルホットスポットとしても機能し、リモート操作とローカル開発という2つのニーズのバランスを取ります。

セキュリティとパフォーマンスの面でも、ESP32のWi-Fi機能は非常に優れた性能を発揮します。

セキュリティ面では、WPA2、WPA3などの主流の暗号化プロトコルや企業レベルの認証をサポートしており、データ伝送のセキュリティを効果的に確保し、ネットワーク攻撃やデータ漏洩を防ぐことができます。

性能面では、最大伝送速度は150Mbpsに達し（一部のハイエンドモデルではさらに高速）、複数の洗練された省電力モードを搭載しており、実際の業務ニーズに応じて消費電力を動的に調整し、バッテリー駆動デバイスのバッテリー寿命を延ばします。また、一部のモデルはマルチアンテナダイバーシティ技術にも対応しており、複雑な環境下での無線信号の安定性と耐干渉性をさらに向上させています。

<https://products.espressif.com/static/Esspressif%20SoC%20Product%20Portfolio.pdf>

Espressif SoC Product Portfolio

Comprehensive Connectivity, AI and HMI SoCs



Product Selector

	Feature	P-Series	S-Series		ESP32-C6		C-Series			H-Series		32-Series
		ESP32-P4	ESP32-S3	ESP32-S2	ESP32-C6	ESP32-C6	ESP32-C61	ESP32-C3	ESP32-C2	ESP32-H4	ESP32-H2	ESP32
Connectivity	Wi-Fi Generation		11b/g/n	11b/g/n	11a/b/g/n/ax/ax	11b/g/n/ax	11b/g/n/ax	11b/g/n	11b/g/n			11b/g/n
	Data Rate		150 Mbps	150 Mbps	150 Mbps	150 Mbps	150 Mbps	150 Mbps	72.2 Mbps			150 Mbps
	Dual Band				✓	✓	✓	✓	✓			
	Wi-Fi 6		✓	✓	✓	✓	✓	✓	✓			✓
	SoftAP		✓	✓	✓	✓	✓	✓	✓			✓
	Sniffing		✓	✓	✓	✓	✓	✓	✓			✓
	WPA Support		WPA3	WPA3	WPA3	WPA3	WPA3	WPA3	WPA3			✓
	Wi-Fi CSI		✓	✓	✓	✓	✓	✓	✓			✓
	Bluetooth Low Energy		5.0		5.3	5.3	5.0	5.0	5.0	5.4	5.0	BT Classic/ 4.2
	LE-Audio									✓		
Radio	Direction Finding									✓		
	Long Range		✓		✓	✓		✓	✓		✓	
	2Mbps Data Rate				✓	✓		✓	✓		✓	
	Extended Advertising				✓	✓		✓	✓		✓	
	BLE-Mesh		✓		✓	✓		✓	✓		✓	
	Thread				✓	✓		✓	✓		✓	
	Zigbee				✓	✓		✓	✓		✓	
	Matter		Wi-Fi		Wi-Fi, Thread	Wi-Fi, Thread	Wi-Fi	Wi-Fi	Wi-Fi	Thread	Thread	Wi-Fi
	Ethernet	✓										✓
	SoC	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
CPU & Memory	Module											
	Frequency		2.4 GHz	2.4 GHz	2.4 GHz 5 GHz	2.4 GHz	2.4 GHz	2.4 GHz	2.4 GHz	2.4 GHz	2.4 GHz	2.4 GHz
	Tx-Power		0 to 20 dBm	0 to 20 dBm	0 to 20 dBm	0 to 20 dBm	0 to 20 dBm	0 to 20 dBm	0 to 20 dBm	-15 to 9 dBm	-15 to 20 dBm	0 to 20 dBm
	Rx-Sensitivity (dBm)		-98.4 (Wi-Fi-IM) -104 (BLE-125k)	-97 (Wi-Fi-IM)	-101(Wi-Fi-IM) -97(Wi-Fi-IM) -107(BLE-125k) -104(802.15.4)	-99.2 (Wi-Fi-IM) -104 (BLE-125k) -104 (802.15.4)	-99.5(Wi-Fi-IM) -105(BLE-125k)	-98.4 (Wi-Fi-IM) -105 (BLE-125k)	-99 (Wi-Fi-IM) -106 (BLE-125k)	- (BLE-125k) - (802.15.4)	-106.5 (BLE-125k) -102.5 (802.15.4)	-98 (Wi-Fi-IM) -94 (BLE-1M)
	CPU	Dual Core RISC-V	Dual Core Xtensa	Single Core Xtensa	Single Core RISC-V	Single Core RISC-V	Single Core RISC-V	Single Core RISC-V	Single Core RISC-V	Dual Core RISC-V	Single Core RISC-V	Single/Dual Core Xtensa
	Clock	400MHz	240MHz	240MHz	240MHz	160MHz	160MHz	160MHz	120MHz	96MHz	96MHz	240MHz
	FPU Extension	✓	✓	✓						✓		✓
	AI Extensions	✓	✓	✓								✓
	LP CPU	40MHz RISC-V	17.5MHz RISC-V	6MHz RISC-V	40MHz RISC-V	20MHz RISC-V						ULP Core
	SRAM	768KB (HP) 16KB (LP)	512KB 16KB (LP)	320KB (HP) 16KB (LP)	384KB (HP) 16KB (LP)	320KB (HP) 16KB (LP)	320KB	400KB 8KB (RTC)	272KB	320KB	320KB 4KB (RTC)	520KB 16KB (RTC)
Security	External Flash	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	PSRAM Support	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Secure Boot	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Flash Encryption	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Crypto Accelerators	RSA, AES, ECC, HMAC	RSA, AES, HMAC	RSA, AES, HMAC	AES, ECC, HMAC, RSA	RSA, AES, ECC, HMAC	ECC,SHA	RSA, AES, HMAC	ECC,SHA	AES, ECC, HMAC	RSA, AES, ECC, HMAC	RSA, AES, SHA
	Digital Signature	ECDSA, RSA	RSA	RSA	ECDSA, RSA	✓	✓	✓	✓	✓	✓	✓
	APM/TEE	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Key Manager	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	MIPSI-CSI	2-lane										
	H.264 Encoder	1080p@30fps										
HMI	MIPSI-DSI	2-lane										✓
	RGB Display	✓	✓	✓								✓
	Camera Interface	✓	✓	✓								✓
	Touch Inputs	14	14	14						14		10
	GPIO Pins	55	45	43	29	30 or 22	22 or 18	22 or 16	14	35	19	34
	SDIO Host	1	1									1
	SDIO Slave				1	1	1					1
	I3C	1										
	SPI	4x SPI 1x LP SPI	4	4	2x SPI 1x GP SPI	2x SPI 1x GP SPI	3	3	3	4	3	4
	USB OTG	1x HS 1x FS	1x FS	1x FS						1	1	
Peripherals	USB Serial	1				1	1	1				
	CAN-FD/TWAI	3x TWAI	1x TWAI	1x TWAI	2x CAN-FD	2x TWAI		1x TWAI		1x CAN-FD	1x TWAI	1x TWAI
	UART	5x UART 1x LP UART	3	2	2	2x UART 1x LP UART	3	2	2	2	2	3
	I2C	2x I2C 2x LP I2C	2	2	1x I2C 1x LP I2C	1x I2C 1x LP I2C	1	1	1	2	2	2
	I2S	3x I2S 1x LP I2S	2	1	1	1	1	1	1	1	1	2
	PWM	1x LED 8ch 2x MCPWM 6ch	1x LED 8ch 2x MCPWM	1x LED 8ch 2x MCPWM	1x LED 6ch 1x MCPWM 6ch	1x LED 6ch 1x MCPWM	1x LED 6ch	1x LED 6ch	1x LED 6ch	1x LED 8ch 2x MCPWM	1x LED 6ch 1x MCPWM	1x LED 16ch 2x MCPWM
	ADC	2x 12 bit	2x 12 bit	2x 12 bit	1x 12 bit 6ch	1x 12 bit 7ch	1x 12 bit	1x 12 bit	1x 12 bit 5ch	1x 12 bit	1x 12 bit 5ch	1x 12 bit 16ch
	DAC			2x 8 bit								2x 8 bit
	Timers, WDT	4x HP OPT, 1x ST, 3x HP WDT, 1x LP OPT, 1x LP WDT, 1x ASWDT	4x OPT, 1x ST, 3x WDT	1x OPT, 1x ST, 3x WDT, 1x XTAL32K WDT	1x ST, 2x OPT, 1x RTCT, 3x DWDT, 1x AWDT	1x ST, 2x OPT, 3x DWDT, 1x AWDT	2x OPT, 1x ST, 2x SWDT, 3x WDT	2x OPT, 3x DWDT, 1x ST	1x OPT, 2x WDT, 1x ST	2x OPT, 1x ST, 3x WDT	2x OPT, 1x ST, 3x WDT	3x WDT, 4x OPT
	Temp Sensor	1	1	1	1	1	1	1	1	1	1	1
	Operating Temperature	-45 to 85	-40 to 105	-40 to 105	-40 to 85	-40 to 105	-40 to 105	-40 to 105	-40 to 105	-40 to 105	-40 to 105	-40 to 125
	Supply Voltage	3.0 to 3.6	3.0 to 3.6	3.0 to 3.6	3.0 to 3.6	3.0 to 3.6	3.0 to 3.6	3.0 to 3.6	3.0 to 3.6	1.7 to 5.0	1.8 to 3.6	2.3/3.0 to 3.6
	Package Type	QFN104	QFN56 LGA56	QFN56	QFN48	QFN40	QFN32 QFN28	QFN28 QFN32	QFN24	QFN52	QFN32	QFN48 LGA48

Wi-Fiプログラミングの一般的な手順

ESP-IDFフレームワークに基づくESP32シリーズチップを使用してWi-Fiアプリケーションを開発する場合、ステーション（STA）クライアントモードの設定が目的であっても、SoftAP（AP）アクセスポイントモードの設定が目的であっても、コアプログラミングプロセスは初期化、設定、接続、イベント処理の3つの主要な段階に従います。

各段階の動作ロジックは非常に普遍的かつ標準化されている。

特定のシナリオ（STAモードでの切断と再接続、APモードでのデバイスアクセス管理など）のプロセス全体を深く理解したい場合は、ESP-IDF公式ドキュメントの「Wi-Fiステーション開発プロセス」および「Wi-Fi AP開発プロセス」に関する詳細なガイドラインを参照してください。

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/wifi-driver/overview.html>

The screenshot shows the ESP-IDF documentation page for the Wi-Fi driver overview. The sidebar on the left contains a table of contents with the following items:

- Reproducible Builds
- RF Calibration
- Standard I/O and Console Output
- Thread Local Storage
- Tools
- Unit Testing in ESP32-S3
- Running ESP-IDF Applications on Host
- USB OTG Console
- USB Serial/JTAG Controller Console
- Wi-Fi Driver
 - Overview
 - ESP32-S3 Wi-Fi Feature List
 - How To Write a Wi-Fi Application
 - ESP32-S3 Wi-Fi API Error Code
 - ESP32-S3 Wi-Fi API Parameter Initialization
 - ESP32-S3 Wi-Fi Programming Model
 - ESP32-S3 Wi-Fi Event Description
 - ESP32-S3 Wi-Fi Configuration
 - Wi-Fi Menuconfig
 - Troubleshooting
 - Station Scenarios
 - Wi-Fi Modes
 - Wi-Fi MAC Protocols
 - Security and Roaming
 - Wi-Fi Performance and Power Save
 - Wi-Fi Vendor Features
 - Wi-Fi Security
 - Wi-Fi Expansion
 - PHY
- Security Guides
- Migration Guides

The main content area is titled "Preparation" and contains the following text:

Generally, the most effective way to begin your own Wi-Fi application is to select an example which is similar to your own application, and port the useful part into your project. It is not a MUST, but it is strongly recommended that you take some time to read this article first, especially if you want to program a robust Wi-Fi application.

This article is supplementary to the Wi-Fi APIs/Examples. It describes the principles of using the Wi-Fi APIs, the limitations of the current Wi-Fi API implementation, and the most common pitfalls in using Wi-Fi. This article also reveals some design details of the Wi-Fi driver. We recommend you to select an **example**.

The following list of examples is provided:

- [wifi/getting_started/station](#) demonstrates how to use the station functionality to connect to an AP.
- [wifi/getting_started/softAP](#) demonstrates how to use the SoftAP functionality to configure ESP32-S3 as an AP.
- [wifi/scan](#) demonstrates how to scan for available APs, configure the scan settings, and display the scan results.
- [wifi/fast_scan](#) demonstrates how to perform fast and all channel scans for nearby APs, set thresholds for signal strength and authentication modes, and connect to the best fitting AP based on signal strength and authentication mode.
- [wifi/wps](#) demonstrates how to use the WPS enrollee feature to simplify the process of connecting to a Wi-Fi router, with options for PIN or PBC modes.
- [wifi/wps_softap_registrar](#) demonstrates how to use the WPS registrar feature on SoftAP mode, simplifying the process of connecting to a Wi-Fi SoftAP from a station.
- [wifi/smart_config](#) demonstrates how to use the smartconfig feature to connect to a target AP using the ESPTOUCH app.
- [wifi/power_save](#) demonstrates how to use the power save mode in station mode.
- [wifi/softap_sta](#) demonstrates how to configure ESP32-S3 to function as both an AP and a station simultaneously, effectively enabling it to act as a Wi-Fi NAT router.
- [wifi/iPerf](#) demonstrates how to implement the protocol used by the iPerf performance measurement tool, allowing for performance measurement between two chips or between a single chip and a computer running the iPerf tool, with specific instructions for testing station/soft-AP TCP/UDP RX/TX throughput.
- [wifi/roaming/roaming_app](#) demonstrates how to use the Wi-Fi Roaming App functionality to efficiently roam between compatible APs.
- [wifi/roaming/roaming_11kvr](#) demonstrates how to implement roaming using 11k and 11v APIs.

1. 初期化フェーズ :Wi-Fi動作のための基本環境を確立する

このフェーズの主な目的は、基盤となるプロトコルスタック、ハードウェアドライバ、およびシステムイベントフレームワークの初期化を完了し、Wi-Fi機能の動作のための強固な基盤を構築することです。具体的な操作は以下のとおりです。

プロトコルスタックとネットワークインターフェースの初期化：

まず、ネットワークデータ伝送を実装するためのコアとなる基盤コンポーネントである軽量TCP/IPプロトコルスタック（LwIP）を初期化します。同時に、ESP32のさまざまなネットワークインターフェース（STA、AP、イーサネットなど）を一元的に管理し、上位レイヤーアプリケーションと基盤となるハードウェア間の重要な橋渡し役となるesp-netifネットワークインターフェース管理モジュールを初期化します。

イベント駆動型フレームワークの構築：

システムのデフォルトイベントループ（esp_event）を作成し、Wi-Fi関連のイベント処理コールバックを登録します。Wi-Fiの状態変化（接続成功、IPアドレス取得、切断など）は非同期イベントとして発生し、イベントループは状態監視と自動処理を実装するためのコアメカニズムとなります。

デフォルトネットワークインターフェースの作成：

ターゲットモード（STA/AP）に基づいて、対応するデフォルトのネットワークインターフェースインスタンス（ESP_NETIF_DEFAULT_WIFI_STAやESP_NETIF_DEFAULT_WIFI_APなど）を作成し、ネットワークインターフェースをWi-Fiハードウェアにバインドして、指定されたインターフェースを介してデータの送受信が可能であることを確認します。

Wi-Fiドライバの初期化：

Wi-Fiハードウェアドライバの初期化を完了するには、esp_wifi_init()インターフェースを呼び出します。この操作により、ESP32内部でWi-Fiタスクスレッドが開始され、無線モジュール、プロトコルスタック、およびハードウェアの適応が完了し、Wi-Fiハードウェアが設定可能かつ動作可能な状態になります。

2. 設定フェーズ :Wi-Fi動作パラメータのカスタマイズ

初期化が完了したら、Wi-Fiのコア動作パラメータを実際の業務要件に合わせて設定し、対象となるネットワーク環境や機能要件に適合させる必要があります。

接続パラメータ設定：

wifi_config_t構造体変数を定義し、モードに対応するコアパラメータを入力します。STAモードでは、ターゲットWi-FiのSSID（ネットワーク名）、パスワード、認証方式（WPA2-PSKなど）を設定する必要があります。APモードでは、ホットスポット名、パスワード、接続可能なデバイスの最大数、認証方式、チャンネルを設定する必要があります。

モード設定:

`esp_wifi_set_mode()` インターフェースを呼び出して、Wi-Fi の動作モード (`WIFI_MODE_STA`、`WIFI_MODE_AP`、`WIFI_MODE_APSTA` など) を明確に設定すると、この操作によって Wi-Fi のコア動作ロジックがロックされます。

設定パラメータが有効になります:

`esp_wifi_set_config()` インターフェースを呼び出して、`wifi_config_t` で設定されたパラメータに対応するネットワークインターフェース (STA/AP) に適用し、パラメータがハードウェアドライバとプロトコルスタックにバインドされるようにします。

3. 接続およびイベント処理フェーズ :Wi-Fiの開始と状態変化の処理

設定が完了したら、Wi-Fi機能を起動し、イベントコールバックを通じて様々な非同期状態を処理することで、Wi-Fi機能の安定した動作を確保します。

Wi-Fi機能の起動:

Wi-Fiモジュールを起動するには、`esp_wifi_start()`インターフェースを呼び出します。この時点で、ESP32は設定されたモードに従って無線モジュールを初期化し、「接続待ち (STA) 」または「ホットスポットブロードキャスト (AP) 」の状態になります。

ネットワーク接続の開始 (STAモードのみ) :

`esp_wifi_connect()` インターフェースを呼び出して、設定済みのターゲット Wi-Fi ネットワークに接続要求を送信します。AP モードでは、この手順は不要で、ホットスポットは自動的にブロードキャストされ、デバイスからのアクセスを待機します。

非同期イベント処理:

`WIFI_EVENT`および`IP_EVENT`イベントコールバック関数を登録することで、STAモードでの「接続成功/失敗」、「IPアドレスの取得」、「接続切断」、APモードでの「デバイスアクセス/切断」など、さまざまな重要な状態を処理できます。

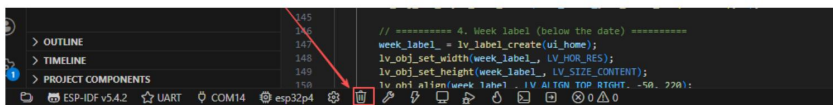
イベント処理は、Wi-Fi機能の堅牢性を確保するための核となる部分であり、例えば、「接続切断」イベントに再接続ロジックを追加したり、「IP取得」イベントでデータ送信タスクをトリガーしたりすることが挙げられます。

上記の専門知識を理解した上で、次にこれら3つのモードについてより深く理解していただけるようご案内します。

プロジェクトコードをコンパイルしてアップロードする前に、プロジェクト内の関連する設定をESP32-C6 Wi-Fiモジュールに合わせて変更し、正しく動作することを確認してください。

(以下のパターンはいずれもESP32-C6 Wi-Fiモジュールを使用するため、いずれか1つが必要です。)

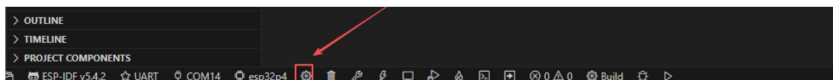
書き込み処理を開始する前に、コンパイル済みのファイルをすべて削除し、プロジェクトを初期状態の「未コンパイル」に戻してください。（これにより、その後のコンパイルが以前の操作の影響を受けないことが保証されます。）



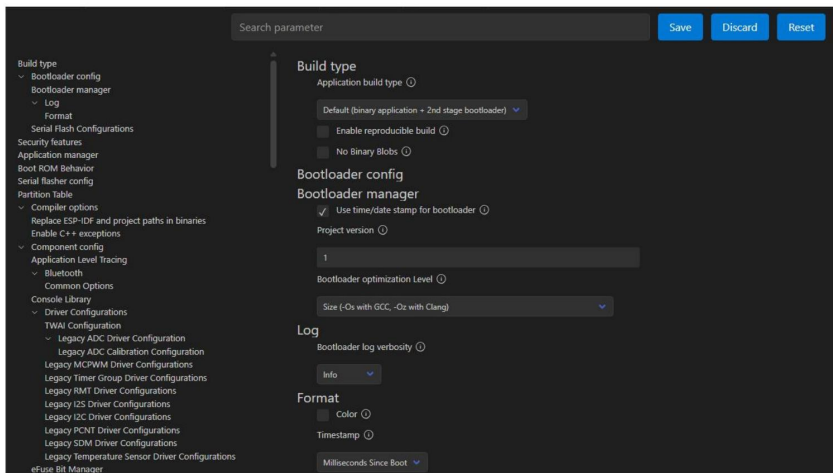
ここでは、最初のセクションの手順に従って、まずESP-IDFのバージョン、コードのアップロード方法、シリアルポート、および使用するチップを選択します。

次に、SDKの設定を行う必要があります。

下の画像にあるアイコンをクリックしてください。

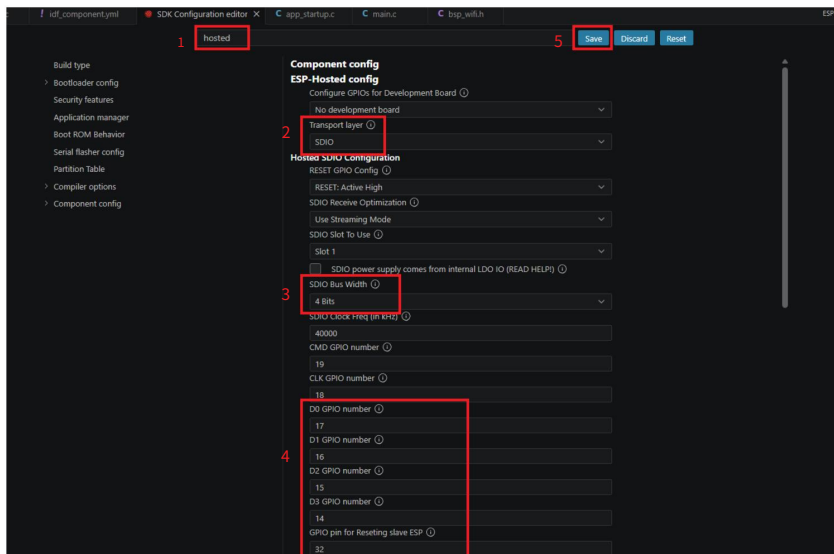


読み込み処理が完了するまでしばらくお待ちください。その後、関連するSDKの設定に進むことができます。

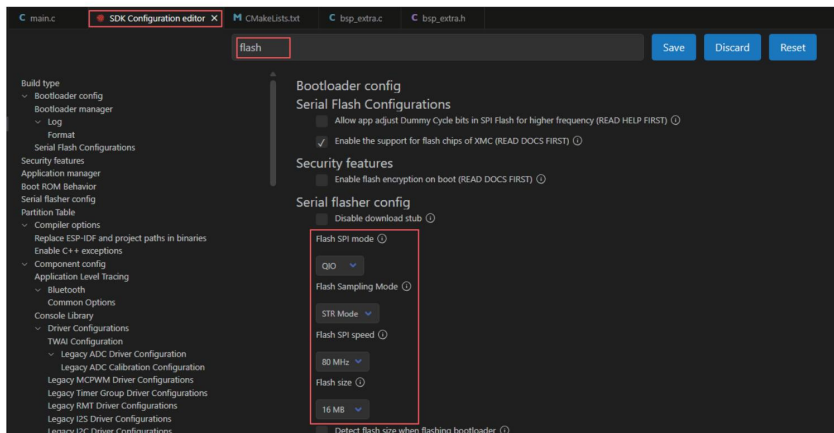


- このプロジェクトでは画面に情報を表示する必要があるため、まずレッスン7のSDK構成を参照して画面表示設定を完了し、画面が正しく機能することを確認することをお勧めします（詳細は101～103ページを参照してください）。

次に、「**ホスト**」に関連する設定項目を検索し、オンボードのESP32-C6のピンマッピングを対応するハードウェアピン構成に変更します。



次に、検索ボックスに「flash」と入力して検索してください。(Flashの設定が私のものと同じであることを確認してください。)



設定が完了したら、必ず設定を保存してください。

ステーション (STA)モード

1. 専門職の定義

ステーション (STA)モードとは、IEEE 802.11ステーションの役割で動作するデバイスを指し、アクティブスキャン (Active Scan)またはパッシブスキャン (Passive Scan)によってアクセスポイント (AP)を検索して関連付け、DHCPプロトコルを介してネットワークパラメータ (IP / ゲートウェイ / DNS)を取得し、最終的にインフラストラクチャネットワークに参加します。

キーワードを一度にすべて暗記する必要はありませんが、それらに精通しておくことは非常に重要です。

2. STAモードは、工学的な観点から具体的にどのような役割を果たしたのでしょうか？

STAモードにおいて、ESP32-C6は実際に一連の処理を完了しました。

- チャンネルスキャン (Channel Scan) 、2.4GHz帯 (チャンネル1~13)
- 認証方式 :WPA2 / WPA3
- 協会
- 重要な合意のための4者間握手
- DHCPリクエスト
- IPレイヤーネットワーク構築の完了

あなたのコードには、たった1つの文しか書かれていませんでした。

```
bsp_wifi_connect(ssid, password);
```

しかし、基盤となるレイヤーは既にWi-Fiプロトコルスタック全体を実行済みです。

3. STAの理解

STAは次のように考えることができます。

- 「私はただの一般ユーザーで、インターネットにアクセスしたいだけです」

その特徴は非常に明確です。

 Wi-Fiなし

 他人の意に関係なく

 関心対象は以下のみ：

- 私が接続したかどうか
- 私がIPアドレスを持っているかどうか
- サーバーにアクセスできるかどうか

4. プロジェクトにおける私たちの役割

ESP32-P4 + ESP32-C6アーキテクチャでは：

C6 (STA)

- ルーターに接続する
- インターネットにアクセスする
- 天気／時刻／雲に関するサービスをリクエストする

P4

- UI/LVGL/ビジネスロジックに重点を置く

これは典型的な「ネットワークコプロセッサ」の設計コンセプトです。

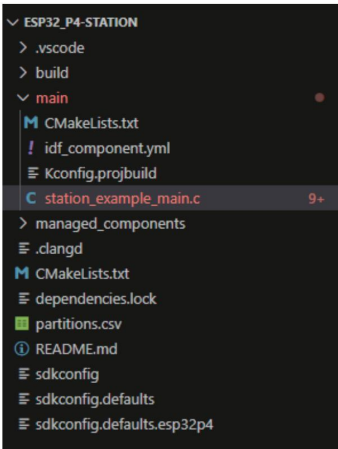
ステーションモードについて理解できたので、それを使って理解を深めていきましょう。

ステーションモードのコードをダウンロードするには、以下のリンクをクリックしてください。

https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson17-Wi-Fi_function

ダウンロード後、VS Codeを使用してステーションモードのコードを開きます。

このコードの枠組みは以下のとおりです。



率直に説明しましょう。使用方法：

メインコードを入力するには、「station_example_main.c」をクリックしてください。

Wi-Fiに接続したい場合は、ここにあるWi-Fi名とパスワードを置き換えてください。

```

9  #include <string.h>
10 #include "freertos/event_groups.h"
11 #include "esp_system.h"
12 #include "esp_wifi.h"
13 #include "esp_event.h"
14 #include "esp_log.h"
15 #include "nvs_flash.h"
16
17 #include "lwip/err.h"
18 #include "lwip/sys.h"
19
20 /* The examples use WiFi configuration that you can set via project configuration menu
21
22 If you'd rather not, just change the below entries to strings with
23 the config you want - ie #define EXAMPLE_WIFI_SSID "mywifissid"
24 */
25 #define EXAMPLE_ESP_WIFI_SSID "elecrow888"
26 #define EXAMPLE_ESP_WIFI_PASS "elecrow2014"
27
28 #define EXAMPLE_ESP_WIFI_MAX_RETRY_COUNT 10
  
```

そうすれば、プロジェクトのコンパイル準備を開始できます。

「ワンクリックアップロード」ボタンをクリックするだけで、コードがコンパイルされ、一度にアップロードされます。アップロードが完了したら、シリアルポートモニターを開いてください。

```

43 #elif CONFIG_ESP_WIFI_AUTH_WEP
44 #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD WIFI_AUTH_WEP
45 #elif CONFIG_ESP_WIFI_AUTH_WPA_PSK
46 #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD WIFI_AUTH_WPA_PSK
47 #elif CONFIG_ESP_WIFI_AUTH_WPA2_PSK
48 #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD WIFI_AUTH_WPA2_PSK
49 #elif CONFIG_ESP_WIFI_AUTH_WPA3_PSK
50 #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD WIFI_AUTH_WPA3_PSK
51 #elif CONFIG_ESP_WIFI_AUTH_WPA2_WPA3_PSK
52 #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD WIFI_AUTH_WPA2_WPA3_PSK
53 #elif CONFIG_ESP_WIFI_AUTH_WAPI_PSK
54 #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD WIFI_AUTH_WAPI_PSK
55 #endif
56 #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD WIFI_AUTH_WAPI_PSK
57 #endif
  
```

Wi-Fi接続が成功したことを示す情報が表示されます。

```

I (3377) esp_cli: Registering command: reboot
I (3377) esp_cli: Registering command: mem-dump
I (3387) esp_cli: Registering command: task-dump
I (3387) esp_cli: Registering command: cpu-dump
I (3397) esp_cli: Registering command: heap-trace
I (3397) esp_cli: Registering command: sock-dump
I (3397) esp_cli: Registering command: host-power-save
I (3407) hci_stub_drv: Host BT Support: Disabled
I (3407) H_SDIO_DRV: Received INIT event
I (3417) H_SDIO_DRV: Event type: 0x22
I (3417) H_SDIO_DRV: Write thread started
I (3807) wifi station: wifi_init_sta finished.
I (3817) RPC_WRAPPER: ESP Event: wifi station started
I (3827) RPC_WRAPPER: ESP Event: wifi station started
I (4847) H_API: esp_wifi_remote_connect
I (6457) RPC_WRAPPER: ESP Event: Station mode: Connected
I (6457) esp_wifi_remote: esp_wifi_internal_reg_rxc: sta: 0x4004b7f2
--- 0x4004b7f2: wifi_sta_receive at J:/My_APP/IDF_MV/v5.4.2/esp-idf/components/esp_wifi/src/wifi_netif.c:38
I (7477) esp_netif_handlers: sta ip: 192.168.50.45, mask: 255.255.255.0, gw: 192.168.50.1
I (7477) wifi station: got ip:192.168.50.45
I (7477) wifi station: connected to ap SSID:elecrow888 password:elecrow2014
I (7477) main_task: Returned from app_main()
  
```

SoftAP (AP)モード

1. 専門職の定義

SoftAP (ソフトウェアアクセスポイント)モードとは、デバイスがソフトウェアを介してWi-Fiアクセスポイント (アクセスポイント) をシミュレートし、定期的にビーコンフレームを送信してSSIDをブロードキャストし、DHCPサーバーおよびローカルゲートウェイとして機能して独立した無線LAN (WLAN)を構築するモードを指します。

2. SoftAPはエンジニアリングにおいてどのような役割を果たしますか？

APモードでは、ESP32-C6は次の動作をします。

定期放送：

- SSID
- 暗号化方式
- チャンネル情報

クライアントとの連携リクエストを承認する

接続されたデバイスごとに：

- IPアドレスを割り当てる
- 接続テーブルを維持する

つまり、

- この時点で、ESP32は「ルーターのように動作する」

3. SoftAPの理解

SoftAPは次のように考えることができます。

- 「ブロードバンド機能のないルーター」

携帯電話の接続を可能にする。

これにより、デバイス間の通信が可能になります。

しかし、デフォルトではインターネットにアクセスしません。

4. 組み込み機器はなぜ特にアクセスポイント (AP)の使用を好むのでしょうか？

なぜなら、それは実際の工学的問題を解決するからです。

- デバイスの電源を初めてオンにしたとき、Wi-Fi の処理方法がわかりません。
パスワード？

答え：

- デバイスでアクセスポイントを有効にする
- 携帯電話をデバイスに接続する
- 実際のWi-Fi情報を入力してください
- NVSに保存
- 再起動 → STAモード

5. アーキテクチャにおける実際のアプリケーション

P4 + C6プロジェクトでは、APモードは通常、以下の目的で使用されます。

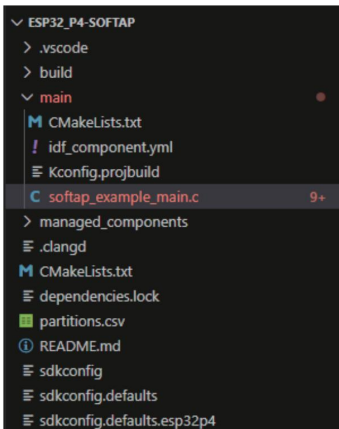
- 流通ネットワーク
- ローカルウェブコントロール
- オフラインデバッグ

SoftAPモードについて理解できたので、それを使って理解を深めていきましょう。

SoftAPモード用のコードをダウンロードするには、以下のリンクをクリックしてください。

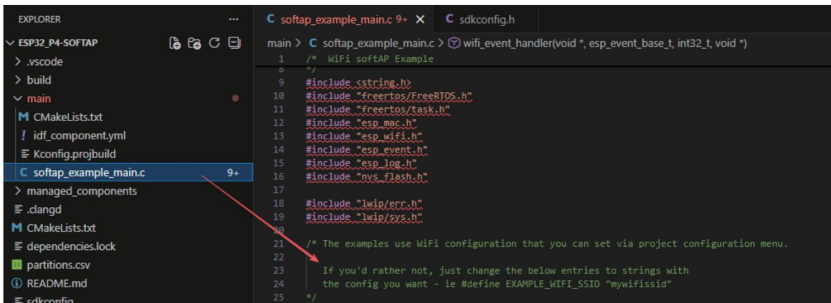
https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson17-Wi-Fi_function

ダウンロード後、VS Codeを使用してSoftAPモードのコードを開きます。このコードのフレームワークは以下のとおりです。



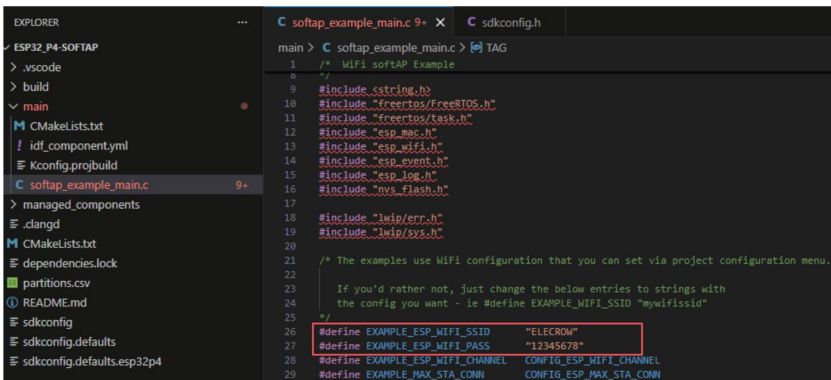
率直に、その使い方を説明しましょう。

softap_example_main.c をクリックし、メインコードを入力してください。



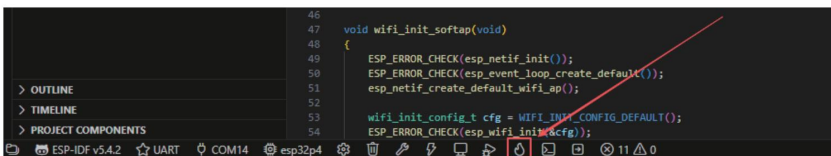
Advance-P4をルーターとして扱うようになったので、コードに設定されているWi-Fiに携帯電話やノートパソコンで接続できるようになります。

コード欄に、ルーターのWi-Fi名とパスワードを入力してください。



そうすれば、プロジェクトのコンパイル準備を開始できます。

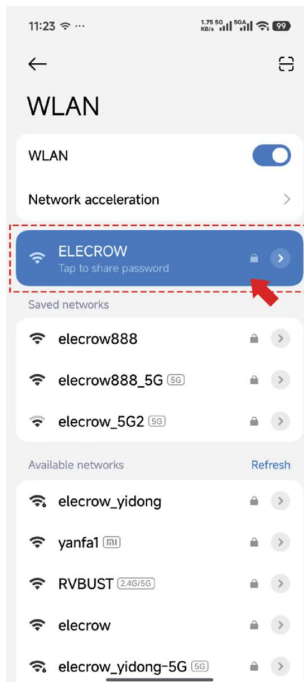
「ワンクリックアップロード」ボタンをクリックするだけで、コードがコンパイルされ、一度にアップロードされます。アップロードが完了したら、シリアルポートモニターを開いてください。



アップロードが完了すると、APモードの準備が完了したことを示す関連情報が表示されます。

```
I (3346) H_API: Transport active
I (3356) transport: Slave chip Id[12]
I (3356) transport: raw_tp_dir[-], flow_ctrl: low[60] high[80]
I (3366) transport: transport_delayed_init
I (3366) esp_cli: Remove any existing deep_sleep cmd in cli
I (3366) esp_cli: Registering command: crash
I (3376) esp_cli: Registering command: reboot
I (3376) esp_cli: Registering command: mem-dump
I (3386) esp_cli: Registering command: task-dump
I (3386) esp_cli: Registering command: cpu-dump
I (3396) esp_cli: Registering command: heap-trace
I (3396) esp_cli: Registering command: sock-dump
I (3396) esp_cli: Registering command: host-power-save
I (3406) hci_stub_drv: Host BT Support: Disabled
I (3406) H_SDIO_DRV: Received INIT event
I (3416) H_SDIO_DRV: Event type: 0x22
I (3416) H_SDIO_DRV: Write thread started
I (3826) wifi softAP: wifi_init_softap finished. SSID:ELECROW password:12345678 channel:1
I (3826) main_task: Returned from app_main()
```

現時点では、お使いの携帯電話またはノートパソコンを使用して、このルーターのWi-Fiに接続してください。



シリアルポートモニターをれば、接続が成功したことが確認でき、携帯電話やノートパソコンのIPアドレスが表示されます。

```

I (3346) H_API: Transport active
I (3356) transport: Slave chip Id[12]
I (3356) transport: raw_tp_dir[-], flow_ctrl: low[60] high[80]
I (3366) transport: transport_delayed_init
I (3366) esp_cli: Remove any existing deep_sleep cmd in cli
I (3366) esp_cli: Registering command: crash
I (3376) esp_cli: Registering command: reboot
I (3376) esp_cli: Registering command: mem-dump
I (3386) esp_cli: Registering command: task-dump
I (3386) esp_cli: Registering command: cpu-dump
I (3396) esp_cli: Registering command: heap-trace
I (3396) esp_cli: Registering command: sock-dump
I (3396) esp_cli: Registering command: host-power-save
I (3406) hci_stub_drv: Host BT Support: Disabled
I (3406) H_SDIO_DRV: Received INIT event
I (3416) H_SDIO_DRV: Event type: 0x22
I (3416) H_SDIO_DRV: Write thread started
I (3826) wifi softAP: wifi_init_softap finished. SSID:ELECROW password:12345678 channel:1
I (3826) main task: Returned from app_main()
I (3836) RPC_WRAP: ESP Event: softap started
I (3856) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (3856) RPC_WRAP: ESP Event: softap started

```

しかし、このWi-Fiにはアクセスできるネットワークがありません。イーサネットケーブルのないルーターのようなものだと考えてください。接続を試みることはできますが、インターネットにアクセスすることはできません。

STA + AP共存モード

1. 専門職の定義

STA + AP共存モード（Station + SoftAP同時接続）とは、単一無線周波数（Single RF）Wi-Fi SoCにおける以下のシナリオを指します。

- ステーションインターフェース（STA）
- SoftAPインターフェース（AP）

これらは仮想ネットワークインターフェイス（仮想Wi-Fiインターフェイス、VIF）を介して同時に実行され、MACレイヤスケジューラ（MACスケジューラ）は、時分割多重（TDM）によって、2つの役割間で送信と受信を時分割方式で切り替えます。

2. 背景となる事実

TA + AP ≠ 2つのWi-Fi モジュール

それよりも：

- 1セットのRF
- PHYセット1つ
- MAC 1セット
- 2つの論理的同一性

ESP32-C6は、プロトコルレベルでは2つのデバイスであるかのように「振る舞う」。

3. プロトコル層とシステム層はどのように実装されていますか？

仮想インターフェース (Virtual Interface,VIF)

ESP-IDFでは,STA + APは実際にはWi-Fiドライバです。

- netif_sta (STAインターフェース)
- netif_ap (APインターフェース)

各インターフェースには以下が含まれます。

- 独立したMACステートマシン
- 独立型IPスタック (lwIP)
- 独立したDHCP動作

しかし、両者は同じ無線周波数ハードウェアを共有している。

4. 時分割多重方式 (TDM)とはどのようなものですか？

アンテナが1つしかないため,ESP32-C6は次のようになります。

- 時には「ルーターの指示に従う」
- 場合によっては「携帯電話にビーコンを送信する」
- クラウドからデータを受信することがある
- 電話からの要求に時々応答する

つまり,Wi-Fiチップが2つのアイデンティティを高速に切り替えていると考えることができます。

この切り替えは、上位層のアプリケーションからは透過的に行われます。

5. チャンネルバインディング

STA + APモードでは,APのチャンネルはSTAが接続されているルーターのチャンネルと同じでなければなりません。理由：

- 単一のRF機器は、2つのチャンネルで同時に動作することはできません。
- STAはAPよりも優先順位が高い。

したがって：

- ルーターはチャンネル6を使用しています。
- SoftAPIはチャンネル6のみで使用できます。

6. STA + AP の理解

デバイスを人間だと考えてみてください。

- STA:

携帯電話を使って会社の本社（外部）に電話をかける

- AP:

同時に、トランシーバーを使えば、近くにいる全員があなたに電話をかけることができます。

一つの行動、二つのコミュニケーション方法。

7. スタンドアロンSTA/APモードとの根本的な違い

スタンドアロンSTAモードでは、ESP32はWi-Fiクライアントとして動作し、外部の既存の無線ネットワークに接続することのみが許可され、その中核となる要件は「外部通信を可能にするためにネットワークにアクセスする」ことです。

スタンドアロンAPモードでは、ESP32自体がWi-Fiホットスポットになることが可能で、その主な要件は「他のデバイスがアクセスできるローカルエリアネットワークを構築すること」です。

STA+AP共存モードは、単一チップ上でこれら2つの動作モードを同時に実現する一方で、その本質は「外部ネットワークにアクセスしてリモート/外部ネットワーク通信を実現する」機能と「ローカルホットスポットを構築してローカルデバイス間の相互作用を可能にする」機能のバランスを取ることにある。

無線周波数時分割多重方式のためスループットは若干低下するものの、リモート操作、ローカルデバッグ／ネットワーク設定など、2つのシナリオを同時に満たすことができ、単一モードの機能範囲を拡大する。

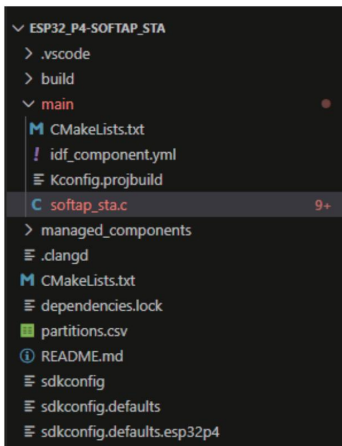
Sta+APモードについて理解できたので、それを使って理解を深めていきましょう。

Sta+APモード用のコードをダウンロードするには、以下のリンクをクリックしてください。

https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson17-Wi-Fi_function

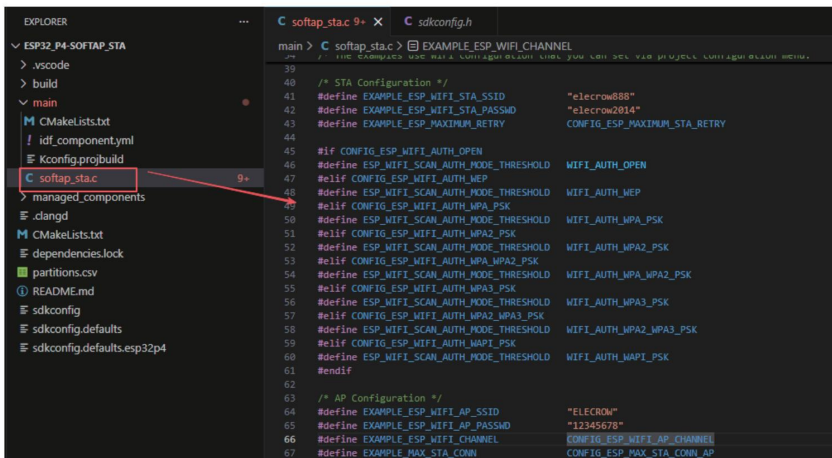
ダウンロード後、VS Codeを使用してSta+APモードのコードを開きます。

このコードの仕組みは以下のとおりです。



率直に、その使い方を説明しましょう。

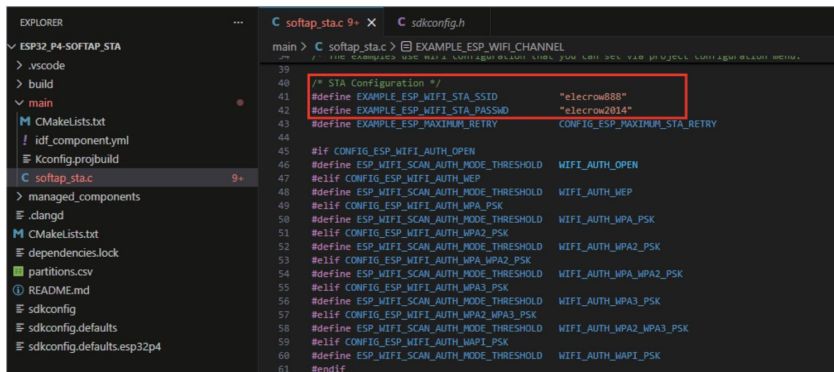
softap_sta.cをクリックし、メインコードを入力してください。



Advance-P4をルーターとして扱うようになったので、コードに設定されているWi-Fiに携帯電話やノートパソコンで接続できるようになりました。

このコードを使えば、Advance-P4を現在使用しているWi-Fiに接続できるだけでなく、Advance-P4自体がルーターとして機能し、他のデバイスも接続できるようになります。

コード欄に接続したいWi-Fiの名前とパスワードを入力してください。

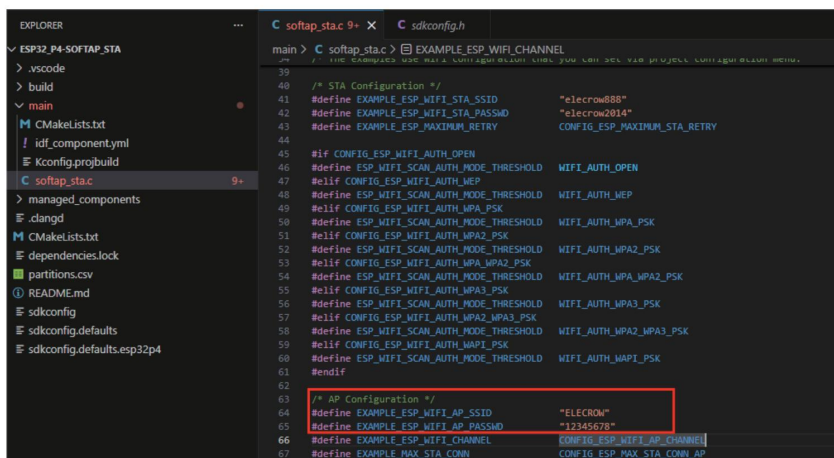


```

39  main > C softap_sta.c > EXAMPLE_ESP_WIFI_CHANNEL
40  /* STA Configuration */
41  #define EXAMPLE_ESP_WIFI_STA_SSID      "elecrow888"
42  #define EXAMPLE_ESP_WIFI_STA_PASSWD    "elecrow2014"
43  #define EXAMPLE_ESP_MAXIMUM_RETRY      CONFIG_ESP_MAXIMUM_STA_RETRY
44
45  #if CONFIG_ESP_WIFI_AUTH_OPEN
46  #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD    WIFI_AUTH_OPEN
47  #elif CONFIG_ESP_WIFI_AUTH_WEP
48  #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD    WIFI_AUTH_WEP
49  #elif CONFIG_ESP_WIFI_AUTH_WPA_PSK
50  #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD    WIFI_AUTH_WPA_PSK
51  #elif CONFIG_ESP_WIFI_AUTH_WPA2_PSK
52  #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD    WIFI_AUTH_WPA2_PSK
53  #elif CONFIG_ESP_WIFI_AUTH_WPA3_PSK
54  #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD    WIFI_AUTH_WPA3_PSK
55  #elif CONFIG_ESP_WIFI_AUTH_WPA2_WPA3_PSK
56  #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD    WIFI_AUTH_WPA2_WPA3_PSK
57  #elif CONFIG_ESP_WIFI_AUTH_WAPI_PSK
58  #define ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD    WIFI_AUTH_WAPI_PSK
59  #endif
60
61  #endif

```

コード欄にルーターのWi-Fi名とパスワードを入力してください。



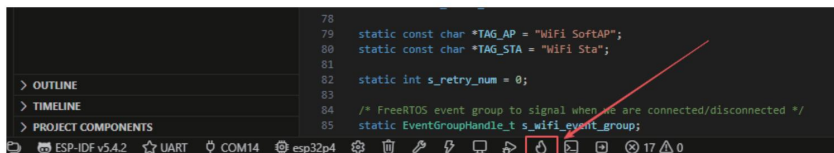
```

62  /* AP Configuration */
63  #define EXAMPLE_ESP_WIFI_AP_SSID      "ELECROW"
64  #define EXAMPLE_ESP_WIFI_AP_PASSWD    "12345678"
65  #define EXAMPLE_ESP_WIFI_CHANNEL      CONFIG_ESP_WIFI_AP_CHANNEL
66  #define EXAMPLE_MAX_STA_CONN          CONFIG_ESP_MAX_STA_CONN
67
68

```

そうすれば、プロジェクトのコンパイル準備を開始できます。

「ワンクリックアップロード」ボタンをクリックするだけで、コードがコンパイルされ、一度にアップロードされます。アップロードが完了したら、シリアルポートモニターを開いてください。



```

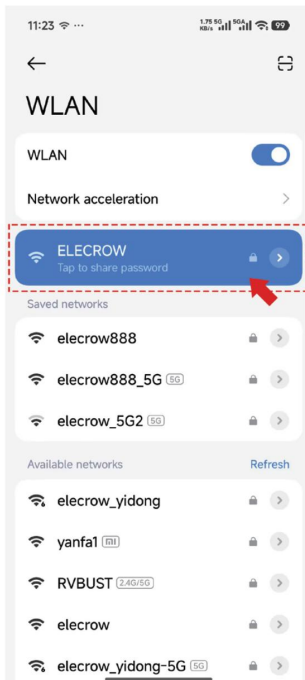
78  static const char *TAG_AP = "Wifi SoftAP";
79  static const char *TAG_STA = "Wifi Sta";
80
81
82  static int s_retry_num = 0;
83
84  /* FreeRTOS event group to signal when we are connected/disconnected */
85  static EventGroupHandle_t s_wifi_event_group;

```

アップロードが完了すると、STAモードが使用しているWi-Fiに正常に接続されたことを示す関連情報が表示されます。

```
I (3408) hci_stub_drv: Host BT Support: Disabled
I (3408) H_SDIO_DRV: Received INIT event
I (3418) H_SDIO_DRV: Event type: 0x22
I (3418) H_SDIO_DRV: Write thread started
I (3728) WiFi SoftAP: ESP_WIFI_MODE_AP
I (3748) WiFi SoftAP: wifi_init softap finished. SSID:ELECROW password:12345678 channel:1
I (3748) WiFi Sta: ESP_WIFI_MODE_STA
I (3768) WiFi Sta: wifi_init_sta finished.
I (3848) RPC_WRAP: ESP Event: wifi station started
I (3848) H_API: esp_wifi_remote_connect
I (3858) RPC_WRAP: ESP Event: softap started
I (3868) RPC_WRAP: ESP Event: wifi station started
I (3878) RPC_WRAP: ESP Event: softap started
I (4888) WiFi Sta: Station started
I (4918) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (7028) RPC_WRAP: ESP Event: Station mode: Connected
I (7028) esp_wifi_remote: esp_wifi_internal_reg_rxcba: sta: 0x4004d6ea
--- 0x4004d6ea: wifi_sta_receive at J:/My_APP/IDF_MY/v5.4.2/esp-idf/components/esp_wifi/src/wifi_netif.c:38
I (8088) esp_netif_handlers: sta ip: 192.168.50.45, mask: 255.255.255.0, gw: 192.168.50.1
I (8088) WiFi Sta: connected to ap SSID:elecrow888 password:elecrow2014
I (8088) main_task: Returned from app_main()
```

その後、携帯電話やノートパソコンを使って、このルーターのWi-Fiに接続することもできます。



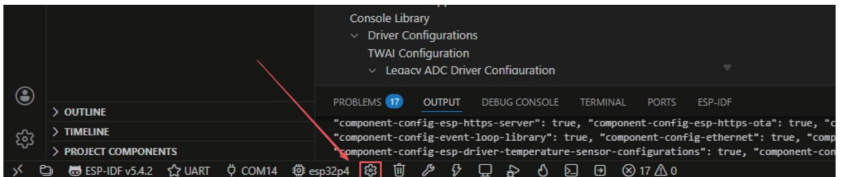
接続が成功すると、シリアルポートモニターに携帯電話のIPアドレスが表示されます。

```
I (3408) hci_stub_drv: Host BT Support: Disabled
I (3408) H_SDIO_DRV: Received INIT event
I (3418) H_SDIO_DRV: Event type: 0x22
I (3418) H_SDIO_DRV: Write thread started
I (3728) WiFi SoftAP: ESP_WIFI_MODE_AP
I (3748) WiFi SoftAP: wifi_init_softap finished. SSID:ELECROW password:12345678 channel:1
I (3748) WiFi Sta: ESP_WIFI_MODE_STA
I (3768) WiFi Sta: wifi_init_sta finished.
I (3848) RPC_WRAP: ESP Event: wifi station started
I (3848) H_API: esp_wifi_remote_connect
I (3858) RPC_WRAP: ESP Event: softap started
I (3868) RPC_WRAP: ESP Event: wifi station started
I (3878) RPC_WRAP: ESP Event: softap started
I (4888) WiFi Sta: Station started
I (4918) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (7028) RPC_WRAP: ESP Event: Station mode: Connected
I (7028) esp_wifi_remote: esp_wifi_internal_reg_rxc: sta: 0x4004d6ea
--- 0x4004d6ea: wifi_sta_receive at J:/My_APP/IDF_MV/v5.4.2/esp-idf/components/esp_wifi/src/wifi_netif.c:38
I (8088) WiFi Sta: Got IP:192.168.50.45
I (8088) esp_netif_handlers: sta ip: 192.168.50.45, mask: 255.255.255.0, gw: 192.168.50.1
I (8088) WiFi Sta: connected to ap SSID:elecrow888 password:elecrow2014
I (8088) main task: Returned from app_main()
I (67118) RPC_WRAP: ESP Event: SoftAP mode: station connected with MAC Addr: c2:ed:6a:29:66:6e
I (67118) WiFi SoftAP: Station c2:ed:6a:29:66:6e joined, AID=1
I (67288) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
```

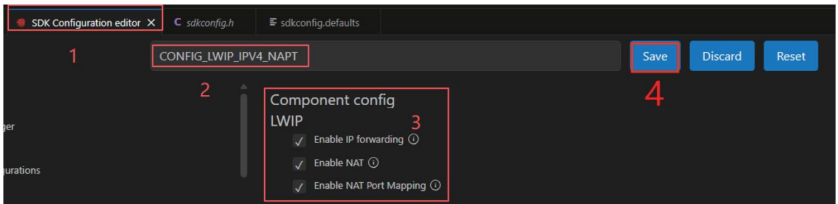
この時点で、スマートフォンのデータ接続をオフにして、設定したAPモードでこのELECROWにのみ接続すれば、簡単にネットワークにアクセスできるようになります。

前提条件は以下のとおりです。

SDK構成にアクセスする必要があります

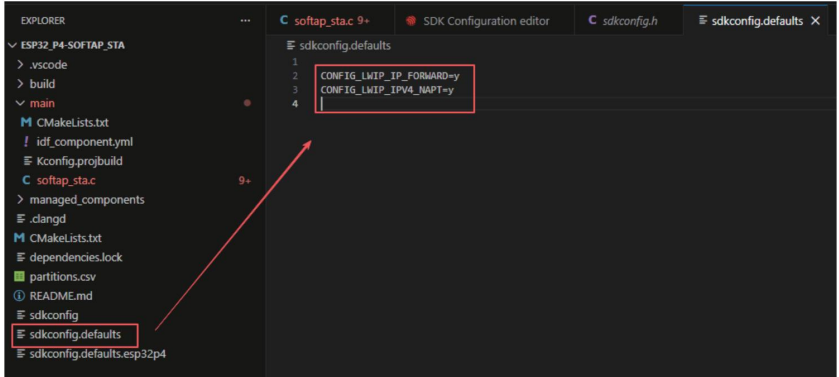


画像に示されている設定を検索して開きます。



このコードをコンパイルした後、ネットワークを簡単に閲覧できるように、スマートフォンをELECROWルーターに接続する必要があります。

私たちのプロジェクトでは、これらの設定は既に準備されています。アップロードするだけで済みます。コード。



なぜこれら2つの設定が有効になっているのですか？

ESP-IDFでは、CONFIG_LWIP_IP_FORWARD=yとCONFIG_LWIP_IPV4_NAPT=yを有効にすることが、Advance-P4 (ESP32-C6ベースのWi-Fiモジュール)がSTA+AP共存モードでクロスインターフェースネットワークデータ転送とネットワークアドレス変換の機能を持つようにするための主要な目的であり、最終的には「Advance-P4 APホットスポットに接続された外部デバイスがAdvance-P4のSTAリンクを介して外部ネットワークにアクセスできる」という主要な要件を満たすことになります。

これら2つの構成の具体的な機能は以下のとおりです。

1. CONFIG_LWIP_IP_FORWARD=y: IPレイヤデータ転送を有効にする

LWIPは、ESP32-C6で使用される軽量TCP/IPプロトコルスタックです。この構成の中核は、プロトコルスタックのIP転送機能を有効にすることです。これにより、ESP32-C6のネットワークプロトコルスタックが、異なるネットワークインターフェース (STAインターフェースとAPインターフェースなど)間でデータパケットを転送できるようになります。

この構成がない場合: STAインターフェース (ユーザーのWiFiに接続)とAPインターフェース (独自のホットスポット)は完全に分離されます。データパケットはそれぞれのインターフェース内でのみ流れることができ、APIに接続されたデバイスはSTA側のネットワークにアクセスできません。

有効化後、ESP32-C6はAPインターフェースから受信した外部ネットワーク要求データパケットをSTAインターフェースに転送し、ユーザーのルーターを介して外部ネットワークにアクセスできます。同時に、STAインターフェースから外部ネットワークから返されたデータパケットをAPインターフェースに転送し、ホットスポットに接続されているデバイスに渡すこともできます。

簡単に言うと、この構成は「インターフェース間通信」を実現するための基礎であり、ESP32-C6において「STAインターフェースとAPインターフェース間のデータパケットチャネルを開く」ことに相当します。

2. CONFIG_LWIP_IPV4_NAPT=y: IPv4ネットワークアドレス変換 (NAT)を有効にします

この構成はIPフォワーディングに基づいており、「異なるサブネット内のデバイス間のアドレス互換性」の問題を解決することを目的としています。その核心は、ネットワークアドレスポート変換 (NAPT)を実装することです。

Advance-P4ホットスポットに接続されたデバイスは、APによって割り当てられたプライベートIPアドレス (192.168.4.xなど)を受け取りますが、Advance-P4のSTAインターフェースは、別のサブネットのユーザールータからプライベートIPアドレスを取得します。外部ネットワークはSTAインターフェースのIPアドレスのみを認識でき、AP側のデバイスのIPアドレスを直接認識することはできません。

NAPTを有効にすると、ESP32-C6はAP側デバイスのIPアドレスとポート番号を自身のSTAインターフェースのIPアドレスとランダムなポート番号に変換し、外部ネットワークにリクエストを送信します。外部ネットワークからデータが返されると、STAインターフェースのIPアドレスとポート番号をAP側デバイスのIPアドレスとポート番号に戻し、データの送信を再開します。

簡単に言うと、この構成により、AP側のデバイスはSTAインターフェースのIPアドレスを「借用」して外部ネットワークにアクセスできるようになり、異なるサブネット間のアドレスマッピングの問題を解決し、AP側のデバイスが正常にインターネットにアクセスするための鍵となります。

2つの構成を組み合わせた最終的な効果

これら2つの設定を有効にすると、Advance-P4は「小型ルーター」としてのコア機能を獲得します。つまり、APホットスポットに接続された携帯電話やコンピューターなどのデバイスは、Advance-P4とローカルで通信できるだけでなく、Advance-P4のSTAリンクを介してユーザーのWiFiのローカルエリアネットワークにアクセスしたり、外部ネットワーク (インターネットへのアクセス、サーバーへの接続など)にアクセスしたりすることもできます。逆に、どちらか一方を無効にすると、AP側のデバイスは通常インターネットにアクセスできなくなります (ローカルでの通信のみが可能になります)。



製作をより簡単に